Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

Graph-Based Strategies for Optimizing Wildfire Suppression

João Aveiro
University of Coimbra
Daniel Neves
University of Coimbra
Paulo Silva
University of Coimbra
Orlando Oliveira
University of Coimbra
Fernando Nogueira
University of Coimbra
Jaime Oliveira da Silva (jaime.silva@fis.uc.pt)
University of Coimbra

Article

Keywords:

Posted Date: June 26th, 2023

DOI: https://doi.org/10.21203/rs.3.rs-3062616/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

Graph-Based Strategies for Optimizing Wildfire Suppression

João Aveiro[1,1], Daniel Neves[1], Paulo Silva[1], Orlando Oliveira[1], Fernando Nogueira[1] and Jaime Oliveira da Silva[1*]

[1*]CFisUC, Department of Physics University of Coimbra, Rua Larga, Coimbra, 3004-516, Portugal.
[2]Departamento de Engenharia Física, Faculdade de Engenharia, Universidade do Porto, Rua Dr. Roberto Frias, s/n, 4200-465 Porto, Portugal.

*Corresponding author(s). E-mail(s): jaime.silva@fis.uc.pt;

Abstract

An algorithm for wildfire suppression based on graph theory and that builds on realistic simulations of the propagation, is efficient, produces results in a short period of time and has the potential to help achieving a faster suppression of a real fire is reported. The suppression algorithm looks for the selection of the best locations to attack the suppression of wildfires, is formally correct and its complexity is computed. The performance of the graph-based algorithm is compared to a random and a brute-force approach. The simulations demonstrate that the graph-based algorithm outperforms the other two and that it delivers excellent results.

1 Introduction

Fires were always a major concern for human societies and it forced the development of strategies to prevent them. In the past decades increasingly devastating fires led to a sharp rise in the effort to prevent and fight them. Indeed, the area burned in wildfires, that can have several natural and human causes, seems to be growing over the years. This is well documented by the statistics published by several countries.

Fires and wildfires have a significant impact in society, affecting human lives, the economy and the climate. They also have become a recurrent phenomenon [1]. So far, the main focus of the scientific community has been on understanding fire propagation and on providing realistic simulations for wildfire propagation [2–5]. The next step is to build tools that can help in a faster suppression of wildfires.

The current work is part of the effort towards the building of software that can help firefighters in their effort to reduce the devastating effects they have in nature. We report on an algorithm for wildfire suppression that builds on realistic simulations of the propagation [5], that is efficient, that produces results in a short period of time and has the potential to help achieving a faster suppression of a real fire. The wildfire suppression algorithm is based on graph theory, and looks for the selection of the best locations to attack the suppression of wildfires. The computer code to run the simulations for the wildfire propagation can be found at [6].

Graph theory is a general framework that is able to describe a large class of physical phenomena from the connectivity patterns in human brain networks [7], the electric conductivity in composites [8], and the computation of the reliability of electric networks [9], to name a few applications. Graph theory was born with the problem of the Königsberg bridges tackled by Euler [10]. Hence, graph theory offers a suitable platform for developing an algorithm for wildfire suppression.

Our wildfire suppression algorithm relies on the semi-empirical Rothermel equations for fire propagation, which are the state of the art approach for surface wildfire simulation [11]. Indeed, the Rothermel model is the standard in Europe, Canada and the United States of America for surface wildfire simulation. The algorithm also requires information on wind that relies on the solution of a 3D wind field model, built by interpolating the data provided by various weather stations arbitrarily located in the region of interest [12]. The wind model is based on a minimisation procedure that was previously developed by others [13].

This work is organised as follows. The algorithms and their complexity are discussed in section 3. The results of the various simulations are reported in the next section, and finally the conclusions are drawn.

2 Results and Discussion

The performance of the Algorithm 1 described in the previous section to achieve the maximal suppression is compared with two other approaches, namely a brute force method and a random suppression scheme. These two algorithms are described below. The algorithms are implemented using the Python programming language and are run over the state-of-the-art wildfire simulator mentioned in the introduction.

In the brute force approach, we remove a single edge and compare the number of burned cells to select the edge which gives the lowest number of

burned cells. The brute force algorithm provides the best solution to the suppression problem given that all the permutations are tested. The brute force algorithm is a NP-hard algorithm. Nevertheless, for small systems, the brute-force approach is feasible and provide a benchmark to compare all the other algorithms considered.

In the random selection approach, the edges to be removed, at each time step, are chosen randomly from the set of propagation steps within the respective time interval.

For the purpose of testing the proposed algorithms, we simulate a realistic wildfire propagation starting from a single ignition cell and propagating for 9 hours. The propagation model used in the simulation is that described in the validation of the simulator [5]. We consider time steps of 1 hour, in which a single edge from the propagation graph can be removed. The number of cells burned at the end of each time interval is used as a metric for the impact of the suppression effort — here represented by the removal of a graph edge — of a proposed approach.

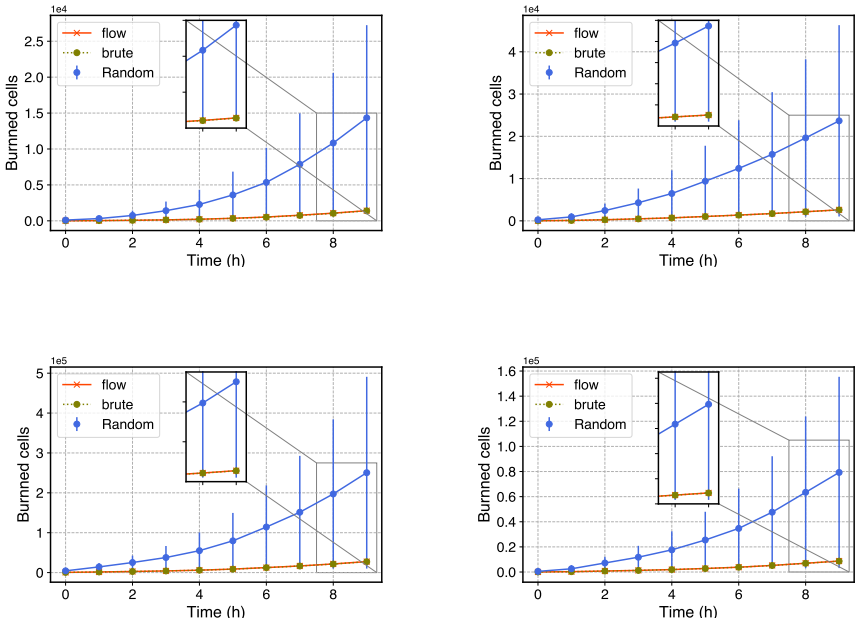

Fig. 1: Burned cells at different propagation time for four different scenarios.

The results of the simulations for the various models are summarised in figure 1. The four different scenarios considered represent different fuel and/or wind models and/or different starting ignition cells for the same raster. The random algorithm had to be executed several times for each scenario in order

to obtain an accurate assessment of the dispersion of its results around the average. The brute force algorithm gives the best solution to the problem at all times, by definition. As shown, the standard deviation of repeated runs of the random algorithm is large and the average solution deviates significantly, already at early times, from the optimal solution given by the brute force algorithm. Although it does not explore all possible solutions, as the brute force method does, the maximum flow algorithm is also able to find the optimal solution at all times. The insertion in figure 1 gives a close-up of the results for the various algorithms at the largest times measured.

Figure 1 shows that we obtain the best results when any of the two deterministic algorithms, flow and brute force, are employed.

In figure 2 and figure 3, we present six scenarios using Algorithm 2 to suppress the fire. In each iteration, we remove a maximum of 150 edges available in the min-cut set. We compare it to Algorithm 1 which called base.

We present the number of burned cells in each iteration. We remark that in the raster approximation for the fire spread simulation in each time step, each cell propagates to 16 new neighbours, so the number of burned cells scales very fast with time.

The line called base corresponds to the propagation when using the max-flow Algorithm 1 for the fire mitigation, and we represent the visualisation of the fired spread by the red scatters called propagation. The min-cut line corresponds to fire suppressed using Algorithm 2, and we see the fire spread as the burned scatter in the blue colour. Finally, we represent the removed edges retrieved from the min-cut in a green scatter called suppressed.

It is possible to observe in figure 2 that Algorithm 2 reduces the burned area when compared with the base propagation, i. e. the propagation where the max-flow edge is removed. Furthermore, this reduction in the burned area depends on the terrain fuel and wind with suppressed edges not following any pattern. Furthermore, there is a dependence on the maximum number of suppressed edges we can remove in the fire propagation, but when we achieve this limit, further increasing it will not change the fire propagation. We can observe the latter in the second row of figure 3, where the change in the burned cells will be meaningless even if we increase the maximum number of suppress edges.

Another case can occur that we can observe in the last two rows of the figure 3. The suppression algorithm (Algorithm 2) can significantly reduce the burned area and, in some cases, stops the propagation as demonstrated in the third row of figure 3.

This last set of tests demonstrates that the suppression depends on the used fuel, terrain and wind. Moreover, it is possible to reduce the fire propagation or even stop it, as demonstrated in figure 3 ,if tackled early. Another point is the place of the suppressed edges; it is possible to observe that choosing the edges from the min-cut promotes cell position that eventually stops or reduces the fire spread. Furthermore, the position of the cells that we suppress is not trivial.

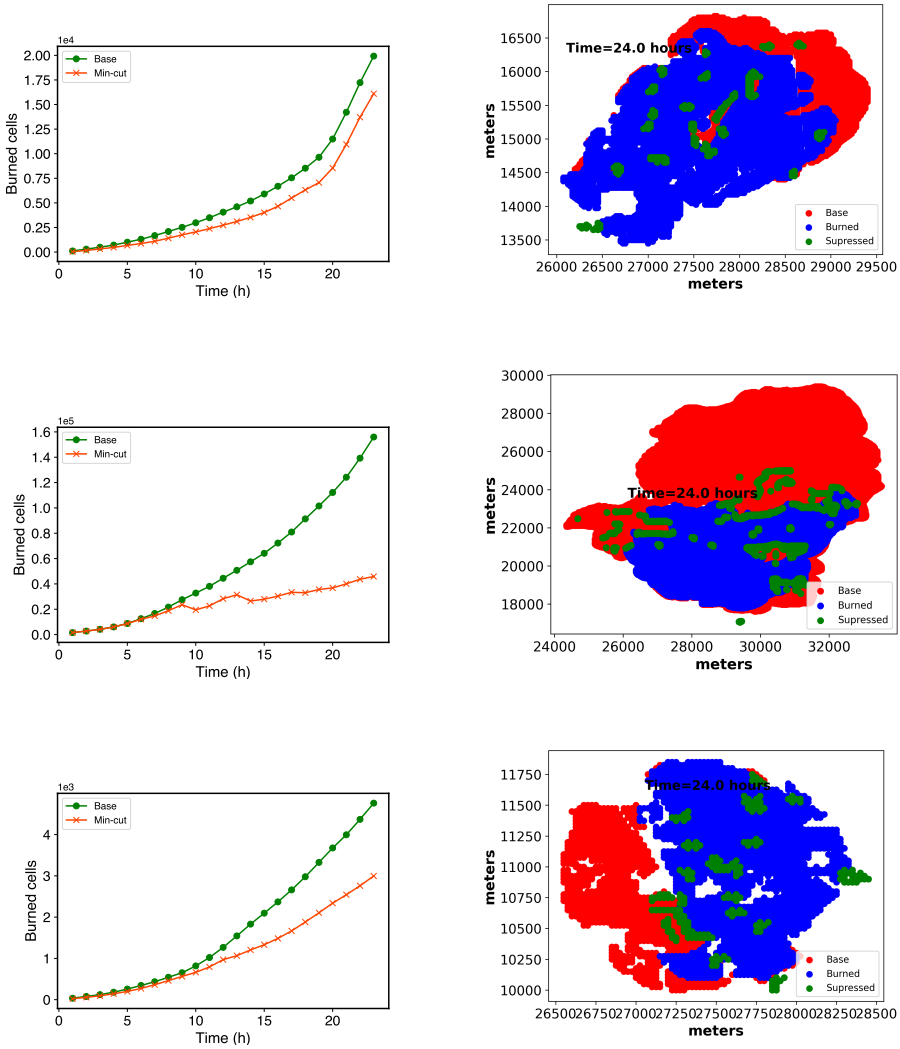

Fig. 2: Fire propagation, for three different scenarios, using Algorithm 2 (min-cut and as blue scatter called burned) and with Algorithm 1 (as a red scatter called Base). We represent the suppressed edges as a green scatter.

Observing the last two rows of figure 3, we can infer that the empiric knowledge of suppressing the wildfire in the initial steps is the best fighting strategy.

We have explored different real scenarios demonstrating Algorithm 2 effectiveness in preventing and stopping fire propagation. The edge capacity, related to the fire propagation time, determines the position of the suppressed edges,

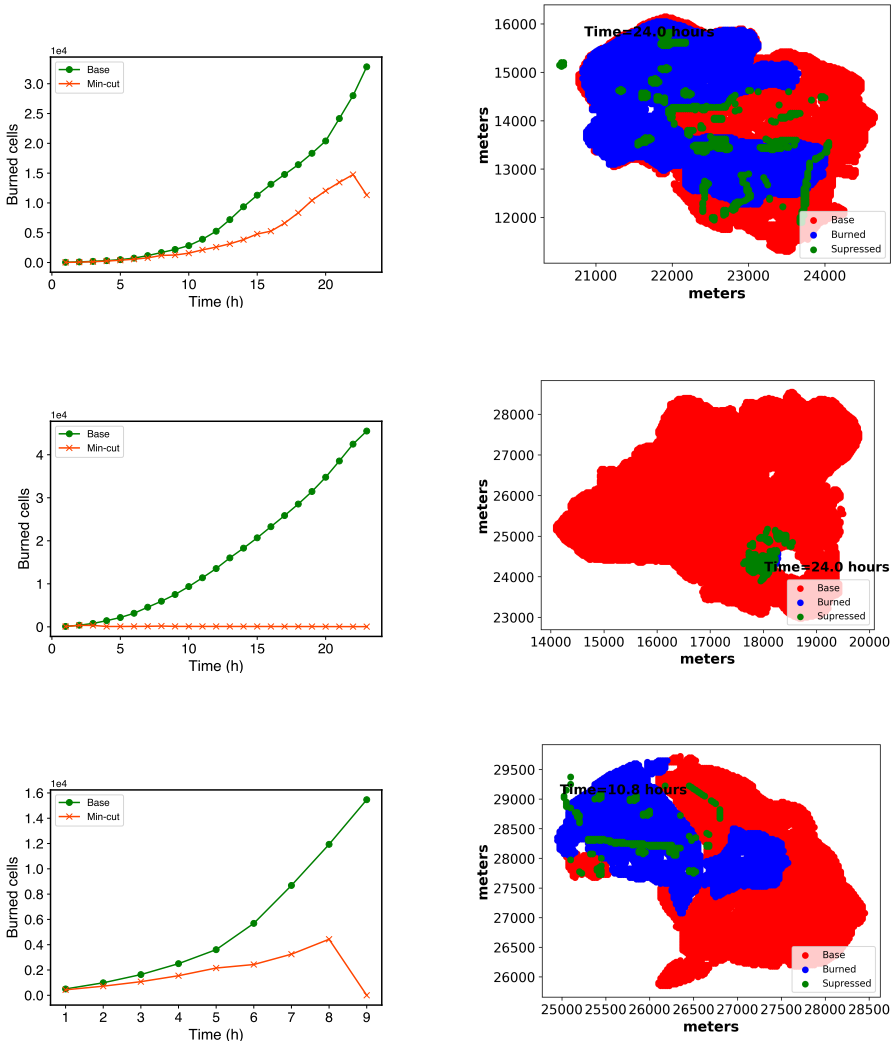

Fig. 3: For three different scenarios, fire propagation uses Algorithm 2 (min-cut and blue scatter called burned) and Algorithm 1 (as a red scatter called Base). We represent the suppressed edges as a green scatter.

and when the fire propagates in a compact form, as in the last two rows of figure 3, it is possible to stop it.

Finally, the results presented here demonstrated the effectiveness of applying graph theory to develop a wild suppression strategy that we can apply in a real-world scenario. Moreover, the suppression strategy is decoupled from the fire simulation since it only needs the time of fire propagation, knowing

that locally we can design a graph that serves as input to our suppression methodologies.

3 Methods

A graph $G(V, E)$ is an ordered pair of vertices, oftentimes called nodes, V, that are connected by edges, E.

A graph can represent physical phenomena by correlating the sets V and E appropriately. Our algorithm uses directed graphs that connect two vertices asymmetrically, where each edge is associated with a non-negative capacity and can receive a flow. A $cut(S, T)$ of a flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$. The capacity is a real-valued function $c : V \times V \to \mathbb{R}$ such that [14]

$$\forall\ u, v \in V,\ c(u, v) \geq 0\,.$$

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following properties:

1. Capacity constrain:$\forall\ u, v \in V, f(u, v) \leq c(u, v)$;
2. Skew symmetry: $\forall\ u, v \in V, f(u, v) = -f(v, u)$;
3. Flow conservation: $\forall\ u \in V \setminus \{s, t\}$ requires $\sum_{v \in V} f(u, v) = 0$.

In our suppression algorithm, each node represents a cell of the propagation raster, i.e., the smallest subdivision considered of the surface area where the fire is propagating. An edge between two nodes represents a valid propagation path with an associated time of propagation (the edge weight). The edge flow capacity is an abstract metric associated with the "ability" of the fire to spread in the corresponding direction. In the algorithm the capacity is taken as the inverse of the edge propagation time: $c = 1/t$. Other definitions that take into account alternative system variables are possible. However, in practice, this definition provided good results. Alternative definitions will be the subject of further studies.

Different algorithms can be built from several mathematical theorems and, in our case, we rely on the max-flow min-cut theorem [14] that can be stated as:

Theorem 1 *(Max flow min-cut) Given a directed graph $G(V, E)$ where $s, t \in V$, the maximal flow value from s to t equals the minimum of the capacities of the cuts separating s from t.*

The algorithm is initialised by specifying a set of ignition vertices, each with a corresponding ignition time. A virtual source vertex, v_{source}, is chosen and connected to each of the ignition vertices by an edge whose weight is its time of ignition. Note that the algorithm uses a directed graph, so the *in degree*,

the number of incoming edges, of the virtual source is set to zero, and the *out degree*, the number of outgoing edges, is initially, the number of ignition cells.

The virtual source is the starting vertex for the first method applied to G, *Dijkstra's shortest path* [14]. After a fixed time, that we take to be one hour, the sub-set of the vertices that can be reached by the fire and the order by which they are reached are identified.

Taking the last N_{sink} vertices reached, a new virtual vertex v_{sink} that will act as a sink for the flow calculation is defined. The weight of each edge is set to the difference between the time it took to reach the sink vertex and an upper time limit defined by the fixed time interval. Note that in this case the *in degree* of the virtual vertex is N_{sink} and the *out degree* is zero. After this procedure, a list of vertices reachable within the selected time frame, a virtual source and sink are used to define the beginning and end of the propagation for the sub-graph.

The reachable sub-graph and the virtual vertices are used to compute the maximum flow with the *Ford-Fulkerson maximum flow* algorithm, taking into account the capacity of the edges. Then, a subset of these edges, which took part of the previously obtained propagation path, is selected. Finally, their capacity and weight are modified or, in an extreme case, removed, simulating the intervention of firefighters.

The shortest path is recomputed resulting in a smaller or, in the worst case scenario, equal sub-graph of reachable vertices. Next, a clean-up operation that removes the virtual vertices and edges is performed. Moreover, the graph can be further simplified by removing all the in-edges of already reached vertices. Finally, a set of the new last-reached vertices are chosen as source nodes for the next time frame and the later procedure is repeated considering this new set. This methodology gives, through the *Ford-Fulkerson* algorithm, the set of edges (i.e. the propagation paths or fire fronts) which should be the main targets of suppression, and the predicted path of propagation from the second run of *Dijkstra's* algorithm.

The selection of a time frame does not guarantee that the optimal solution is achieved. However, it allows the usage of smaller sub-graphs for a step-by-step determination of the almost-ideal cut. Similarly, it mimics the actual suppression methods.

For a starting time t_0, an upper time limit t_{max}, and a time step interval of Δt, the total number of discrete time steps considered is $N_{\text{steps}} = \left\lceil \frac{t_{max}-t_0}{\Delta t} \right\rceil$.

Algorithm 1 Calculates the optimal edge to suppress a wildfire

Require: $G(V, E)$, W(E), t_0, t_{max}, Δt
Ensure: $e_{supressed} \in E$

```
R ← 1                                          ▷ Number of nodes reached
t ← t_0
for e ∈ E(G) do c(e) ← 1/W(e) ▷ O(‖E‖)         ▷ Initialization of capacities
end for
v_source → V(G) ▷ O(1)
v_sink → V(G) ▷ O(1)
while t < t_max do
    t ← t + Δt ▷ O(1)
    c(e) ∈ [v_source, v], c(e) ∈ ∅ ▷ O(1)
    T, P ← ShortestPath(G, v_source, t) ▷ O(‖E‖ log ‖V‖)
    if ‖P‖ = R then continue
    end if
    R = ‖P‖
    v_∈P ∪ v_sink∈V → e ∈ E ▷ t − T[v] ▷ O(1)
    F ← MaxFlow(G, v_source, v_sink)▷ O(‖V‖‖E‖²)
    e_max ∈ F → ∅ ▷ O(‖E‖)
    T, P ← ShortestPath(G, v_source, t)▷ O(‖E‖ log ‖V‖)
    GraphSimplification(G, T, P) ▷ O(‖E‖)
end while
```

The algorithm described is summarised in Tab. 1. Its complexity can be computed:

- The Max Suppression Algorithm has an order of maximum execution time of $O(N_{\text{steps}}\|V\|\|E\|^2)$.
  Observing the designed algorithm with the associated computational costs, we can infer that the execution time is limited by the max flow computational cost ($O(\|V\|\|E\|^2)$). Moreover the max flow suppression algorithm calls the max flow N_{steps} times hence the associated computational cost is $O(N_{\text{steps}}\|V\|\|E\|^2)$.
- The algorithm ends when $t \geq t_{max}$.
  By inspection, t ends its incrementation when $t > t_{max}$ in the *while* cycle, hence the *while* cycle is finite with t_{max} being the variable that controls the number of iterations.

Although the maximum flow algorithm considers the removal of a single edge, the algorithm can be generalised straightforwardly to include the removal of several edges. In this case, Theorem 1 can still be used with the results of the modified max flow procedure modified to retrieve a min-cut. We stress that the min-cut is a set of edges that disconnects the graph. This set of edges stops the wildfire propagation when they are all removed. Moreover, the empirical

knowledge of the firefighters, conjugated with the wind and terrain behaviour, is needed to select the edges from the min-cut to be removed.

Modifying the initial algorithm to retrieve the edges that belong to the min-cut, we can remove a set of edges to stop the propagation. Finally, we formally describe the last modification in Algorithm 2.

Algorithm 2 Calculates the minimum cut to suppress a wildfire

1: Require $G(V, E)$, W(E), t_0, t_{max}, Δt
Ensure: $e_{supressed} \in E$
2: $R \leftarrow 1$ ▷ Number of nodes reached
3: $t \leftarrow t_0$
4: **for** $e \in E(G)$ **do** $c(e) \leftarrow 1/W(e)$ ▷ $O(\|E\|)$ ▷ Initialization of capacities
5: **end for**
6: $v_{source} \rightarrow V(G)$ ▷ $O(1)$
7: $v_{sink} \rightarrow V(G)$ ▷ $O(1)$
8: **while** $t < t_{max}$ **do**
9: $t \leftarrow t + \Delta t$ ▷ $O(1)$
10: $c(e) \in [v_{source}, v], c(e) \in \emptyset$ ▷ $O(1)$
11: T, $P \leftarrow ShortestPath(G, v_{source}, t)$ ▷ $O(\|E\| \log \|V\|)$
12: **if** $\|P\| = R$ **then** continue
13: **end if**
14: $R = \|P\|$
15: $v_{\in P} \cup v_{sink \in V} \rightarrow e \in E$ ▷ $t - T[v]$ ▷ $O(1)$
16: $MinCut \leftarrow MaxFlow(G, v_{source}, v_{sink})$▷ $O(\|V\|\|E\|^2)$
17: **while** $i < N_{suppress}$ **do**
18: $e \in MinCut \rightarrow \varnothing$ ▷ $O(\|E\|)$
19: $i \leftarrow i + 1$ ▷ $O(1)$
20: **end while**
21: T, $P \leftarrow ShortestPath(G, v_{source}, t)$▷ $O(\|E\| \log \|V\|)$
22: **end while**

The complexity analyses of Algorithm 1 also apply to Algorithm 2. Until line 16, the two Algorithms share the same structure. The difference between the two is that for Algorithm 2 we introduce a new variable called $N_{suppress}$ that controls the number of edges removed that we chose from the Min-Cut set.

4 Conclusions

The focus of this work is to deliver a first attempt to establish a fast enough software that helps to reduce the consequences of wildfires. In the past, the focus was on wildfire simulation. The algorithms discussed are based on the Rothermel equations that provide simulations of fire spread in good agreement

with the observation of natural wildfires, when the wind and fuel terrain are known.

The suppression algorithm discussed is founded in graph theory. Its performance is compared with two different approaches, a random and brute force algorithm. Brute force, being NP-hard, is impractical for medium to large systems, but, when applicable, it finds the optimal solution. Moreover, we are able to show that new algorithm usually finds the optimal solution in polynomial time.

The min-cut set of edges obtained by the use of this algorithm defines a direct and concrete approach for the suppression of a wildfire. Despite this, the cardinality of this set might exceed the capabilities of the suppression efforts available in real events. As such, in future work it would be interesting to further study the effective selection of a sub-set of the min-cut edges, possibly employing some heuristic approach.

Nevertheless, we expect that this algorithm will help the firefighters determine the best options for tackling fire propagation.

5 Acknowledgements

This work was carried out under the project IMFire - Intelligent Management of Wildfires,
ref. PCIF/SSI/0151/2018, and was fully funded by national funds through the FCT - Fundação para a Ciência e a Tecnologia, I.P. Investigations done at CFisUC were supported by FCT - Fundação para a Ciência e a Tecnologia, Portugal (projects UIDB/04564/2020 and UIDP/0456/2020. J. Oliveira da Silva thanks FCT - contract CEECIND/00579/2017.
P. J. Silva acknowledges financial support from FCT (Portugal) under Contract No. CEECIND/00488/2017. We thank the Laboratory for Advanced Computing (LCA) of the University of Coimbra for the computer time allocated to this project.

References

[1] Moritz, M.A., Batllori, E., Bradstock, R.A., Gill, A.M., Handmer, J., Hessburg, P.F., Leonard, J., McCaffrey, S., Odion, D.C., Schoennagel, T., Syphard, A.D.: Learning to coexist with wildfire. Nature **515**, 58–66 (2014). https://doi.org/10.1038/nature13946

[2] Beer, T., Enting, I.G.: Fire spread and percolation modelling. Mathematical and Computer Modelling **13**, 77–96 (1990). https://doi.org/10.1016/0895-7177(90)90065-U

[3] Khelloufi, K., Baara, Y., Clerc, J.P., Porterie, B., Zekri, N.: Forest fire spread with non-universal critical behavior. Journal of Statistical Mechanics: Theory and Experiment **2013**, 10027 (2013). https://doi.org/10.1088/1742-5468/2013/10/P10027

[4] Abades, S.R., Gaxiola, A., Marquet, P.A.: Fire, percolation thresholds and the savanna forest transition: a neutral model approach. Journal of Ecology **102**, 1386–1393 (2014). https://doi.org/10.1111/1365-2745.12321

[5] Lopes, A.M.G., Cruz, M.G., Viegas, D.X.: Firestation an integrated software system for the numerical simulation of fire spread on complex topography. Environmental Modelling and Software **17**, 269–285 (2002). https://doi.org/10.1016/S1364-8152(01)00072-X

[6] Aveiro, J., Neves, D., da Silva, J.O. https://github.com/PyFireCoimbra/PyFireStation

[7] Farahani, F.V., Karwowski, W., Lighthall, N.R.: Application of graph theory for identifying connectivity patterns in human brain networks: A systematic review. Frontiers in Neuroscience **13** (2019). https://doi.org/10.3389/fnins.2019.00585

[8] Silva, J., Simoes, R., Lanceros-Mendez, S., Vaia, R.: Applying complex network theory to the understanding of high-aspect-ratio carbon-filled composites. EPL (Europhysics Letters) **93**, 37005 (2011). https://doi.org/10.1209/0295-5075/93/37005

[9] Silva, J., Gomes, T., Tipper, D., Martins, L., Kounev, V.: An effective algorithm for computing all-terminal reliability bounds. Networks **66**, 282–295 (2015). https://doi.org/10.1002/net.21634

[10] Euler, L.: Solutio problematis ad geometriam situs pertinentis. Commentarii academiae scientiarum Petropolitanae **8**, 128–140 (1741)

[11] Rothermel, R.C.: A mathematical model for predicting fire spread in wildland fuels. Res. Pap. INT-115. Ogden, UT: U.S. Department of Agriculture, Intermountain Forest and Range Experiment Station. 40 p. (1972)

[12] Ross, D.G., Smith, I.N., Manins, P.C., Fox, D.G.: Diagnostic wind field modeling for complex terrain: Model development and testing. Journal of Applied Meteorology **27**, 785–796 (1988). https://doi.org/10.1175/1520-0450(1988)027⟨0785:DWFMFC⟩2.0.CO;2

[13] Sasaki, Y.: An objective analysis based on the variational method. Journal of the Meteorological Society of Japan. Ser. II **36**, 77–88 (1958). https://doi.org/10.2151/jmsj1923.36.3_77

[14] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms, Third Edition, 3rd edn. The MIT Press, Cambridge (2009)

6 Author contributions

JA, DN and JOS wrote the article, designed the algorithms and ran the simulations. PS, OO and FN wrote the article and designed the algorithms.

7 Data availability

Derived data supporting the findings of this study are available from the corresponding author JOS on request.

8 Figure legends

1. Burned cells at different propagation time for four different scenarios.
2. Fire propagation, for three different scenarios, using Algorithm 2 (min-cut and as blue scatter called burned) and with Algorithm 1 (as a red scatter called Base). We represent the suppressed edges as a green scatter.
3. For three different scenarios, fire propagation uses Algorithm 2 (min-cut and blue scatter called burned) and Algorithm 1 (as a red scatter called Base). We represent the suppressed edges as a green scatter